

Air University



Cryptography (Lab)

Year 2026

Mr. Omer Ali

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: February 26, 2026

Lab Task # 02

“Performing Brute-Force On S-Des to Find Key”

1. Code

```
#include <iostream>
#include <string>
#include <bitset>
#include <cstdint>
#include <vector>

using namespace std;

// Permutation Tables
// All indices are 1-based (as in the SDES specification)
const int P10[10] = {3, 5, 2, 7, 4, 10, 1, 9, 8, 6};
const int P8[8] = {6, 3, 7, 4, 8, 5, 10, 9};
const int IP[8] = {2, 6, 3, 1, 4, 8, 5, 7};
const int IP_INV[8] = {4, 1, 3, 5, 7, 2, 8, 6};
const int EP[8] = {4, 1, 2, 3, 2, 3, 4, 1}; // Expansion/Permutation
const int P4[4] = {2, 4, 3, 1};

// S-boxes (4x4)
const int S0[4][4] = {
    {1, 0, 3, 2},
    {3, 2, 1, 0},
    {0, 2, 1, 3},
    {3, 1, 3, 2}};

const int S1[4][4] = {
    {0, 1, 2, 3},
    {2, 0, 1, 3},
    {3, 0, 1, 0},
    {2, 1, 0, 3}};

// ===== Utility Functions =====

// Check if a string contains only '0' and '1'
bool isBinary(const string &s)
{
    for (char c : s)
        if (c != '0' && c != '1')
            return false;
    return true;
}

// Read a binary string of exactly 'bits' characters, with validation
string readBinary(int bits)
{
    string input;
    while (true)
```

```
{
    cin >> input;
    if (input.length() == bits && isBinary(input))
        return input;
    cout << "Invalid input. Enter exactly " << bits << " bits (0/1): ";
}

// Apply a permutation table to an input value.
uint16_t permute(uint16_t input, int in_len, const int perm[], int out_len)
{
    uint16_t output = 0;
    for (int i = 0; i < out_len; i++)
    {
        // Source bit position (0-based from the left)
        int src = perm[i] - 1;
        // Extract that bit (counting from MSB)
        uint16_t bit = (input >> (in_len - 1 - src)) & 1;
        // Place it in the output (MSB first)
        output |= (bit << (out_len - 1 - i));
    }
    return output;
}

// Circular left shift on a 5-bit value
uint8_t leftShift5(uint8_t val, int shifts)
{
    shifts %= 5;
    val &= 0x1F; // keep only lower 5 bits
    return ((val << shifts) | (val >> (5 - shifts))) & 0x1F;
}

// ===== Key Schedule =====
void generateSubkeys(uint16_t key, uint8_t &k1, uint8_t &k2)
{
    // Step 1: P10
    uint16_t permuted = permute(key, 10, P10, 10);

    // Split into two 5-bit halves
    uint8_t left = (permuted >> 5) & 0x1F;
    uint8_t right = permuted & 0x1F;

    // Round 1 (shift 1)
    left = leftShift5(left, 1);
    right = leftShift5(right, 1);
    uint16_t combined = (left << 5) | right;
    k1 = permute(combined, 10, P8, 8); // first subkey

    // Round 2 (shift 2 more → total 3)
    left = leftShift5(left, 2);
    right = leftShift5(right, 2);
    combined = (left << 5) | right;
```

```
k2 = permute(combined, 10, P8, 8); // second subkey
}

// ===== S-box Lookup =====
// Input: 4 bits (packed in low nibble of val)
// Output: 2 bits (result of S-box)
uint8_t sbox(uint8_t val, const int box[4][4])
{
    // Row: bits 1 and 4 (1-based) → bits 4 and 1 (0-based)
    uint8_t row = ((val & 0x8) >> 2) | (val & 0x1);
    // Column: bits 2 and 3
    uint8_t col = (val >> 1) & 0x3;
    return box[row][col] & 0x3;
}

// ===== Feistel Round Function =====
// f(R, SK) : R is 4 bits, SK is 8 bits (subkey)
uint8_t f(uint8_t r, uint8_t sk)
{
    // Expansion/Permutation (4 → 8 bits)
    uint8_t expanded = permute(r, 4, EP, 8);

    // XOR with subkey
    uint8_t xored = expanded ^ sk;

    // Split into two 4-bit halves for the S-boxes
    uint8_t left = (xored >> 4) & 0x0F;
    uint8_t right = xored & 0x0F;

    // Apply S-boxes (each gives 2 bits)
    uint8_t s0_out = sbox(left, S0);
    uint8_t s1_out = sbox(right, S1);

    // Combine (4 bits: s0_out as high 2, s1_out as low 2)
    uint8_t combined = (s0_out << 2) | s1_out;

    // Apply P4 permutation
    return permute(combined, 4, P4, 4);
}

// ===== Encryption =====
uint8_t encrypt(uint8_t plain, uint8_t k1, uint8_t k2)
{
    // Initial Permutation
    uint8_t ip = permute(plain, 8, IP, 8);

    // Split into left and right halves (4 bits each)
    uint8_t left = (ip >> 4) & 0x0F;
    uint8_t right = ip & 0x0F;

    // Round 1
    uint8_t temp = right;
```

```
right = left ^ f(right, k1);
left = temp;

// Round 2
temp = right;
right = left ^ f(right, k2);
left = temp;

// After the two rounds, swap the halves (as required by Feistel)
uint8_t combined = (right << 4) | left;

// Inverse Initial Permutation
return permute(combined, 8, IP_INV, 8);
}

// ===== Decryption =====
// Same encryption function but with subkeys reversed
uint8_t decrypt(uint8_t cipher, uint8_t k1, uint8_t k2)
{
    return encrypt(cipher, k2, k1);
}

// ===== Brute Force Function =====
void bruteForce(const string &plainStr, const string &cipherStr, uint16_t originalKey)
{
    uint8_t plain = stoi(plainStr, nullptr, 2);
    uint8_t targetCipher = stoi(cipherStr, nullptr, 2);

    cout << "\nStarting brute force over all 1024 possible keys...\n\n";

    vector<uint16_t> matchingKeys;

    for (uint16_t key = 0; key < 1024; ++key)
    {
        // Generate subkeys for this key
        uint8_t k1, k2;
        generateSubkeys(key, k1, k2);

        // Encrypt the plaintext
        uint8_t cipher = encrypt(plain, k1, k2);

        // Check if it matches the target ciphertext
        bool match = (cipher == targetCipher);

        // Display the attempt
        cout << "Key " << key << " (binary " << bitset<10>(key) << ") -> ";
        cout << "Ciphertext: " << bitset<8>(cipher);
        if (match)
        {
            cout << " <--- MATCH";
            if (key == originalKey)
                cout << " (ORIGINAL KEY)";
        }
    }
}
```

```
        matchingKeys.push_back(key);
    }
    cout << endl;
}

// Report results
cout << "\n==== BRUTE FORCE COMPLETE =====\n";
if (matchingKeys.empty())
{
    cout << "No key found that maps the given plaintext to the given ciphertext.\n";
}
else
{
    cout << "Found " << matchingKeys.size() << " matching key(s):\n";
    for (uint16_t k : matchingKeys)
    {
        cout << "    " << bitset<10>(k) << " (decimal " << k << ")\n";
        if (k == originalKey)
            cout << "    <--- ORIGINAL KEY\n";
        cout << "\n";
    }
}
}

// ===== Main Program =====
int main()
{
    cout << "\n=== SIMPLIFIED DES (SDES) ===\n";

    // Read plaintext and key
    cout << "Enter 8-bit plaintext: ";
    string plainStr = readBinary(8);
    cout << "Enter 10-bit key: ";
    string keyStr = readBinary(10);

    // Convert to integers
    uint8_t plain = stoi(plainStr, nullptr, 2);
    uint16_t key = stoi(keyStr, nullptr, 2);

    // Generate subkeys
    uint8_t k1, k2;
    generateSubkeys(key, k1, k2);

    // Encrypt
    uint8_t cipher = encrypt(plain, k1, k2);
    uint8_t decrypted = decrypt(cipher, k1, k2);

    // Display encryption results
    cout << "\nSubkey K1: " << bitset<8>(k1);
    cout << "\nSubkey K2: " << bitset<8>(k2);

    cout << "\n\nPlaintext : " << plainStr;
```

```
cout << "\nCiphertext: " << bitset<8>(cipher);
cout << "\nDecrypted : " << bitset<8>(decrypted);

if (plain == decrypted)
    cout << "\n\nDecryption successful - original plaintext recovered.\n";
else
    cout << "\n\nDecryption failed - something went wrong.\n";

// Now perform brute force using the same plaintext and resulting ciphertext
bruteForce(plainStr, bitset<8>(cipher).to_string(), key);

cout << endl;
return 0;
}
```

Explanation:

- Fixed Permutation Tables:** The code features constant arrays (P10, P8, IP, EP, etc.) that act as templates for rearranging bits. These tables follow the standard S-DES specifications to guarantee consistent bit transformations throughout the process.
- Bitwise Permutation Engine:** The `permute` function serves as a handy utility that rearranges the bits of an input value according to a specified table. It accomplishes this using bitwise shifts (`>>`, `<<`) and the bitwise OR (`|`) operator.
- Key Schedule Logic:** The `generateSubkeys` function processes an initial 10-bit key by first applying the P10 permutation. It then divides the result into two 5-bit halves, performs circular left shifts, and applies P8 to produce the two 8-bit subkeys (K1 and K2) required for the two rounds of encryption.
- S-Box Substitution:** The `sbox` function captures the non-linear aspect of the cipher. By using the 1st and 4th bits of a 4-bit nibble to determine the row, and the 2nd and 3rd bits to locate the column in a 4x4 matrix, this function returns a 2-bit result.
- The Feistel Round (f):** This function carries out the encryption process, expanding the rightmost 4 bits of the data to 8 bits using EP. It then performs an XOR operation with a subkey, processes the output through the S-boxes, and applies a P4 permutation.
- Encryption and Decryption Flow:** The `encrypt` function executes the Initial Permutation, followed by two rounds of Feistel (with a swap in between), and concludes with the Inverse Initial Permutation. Meanwhile, the `decrypt` function invokes `encrypt`, but inverts the order of K1 and K2 for decryption.
- Input Handling and Validation:** The `readBinary` function ensures that user inputs are valid binary strings (consisting only of 0s and 1s) and have the correct lengths (either 8 or 10 bits). This minimizes the chances of conversion errors.
- Brute Force Implementation:** The `bruteForce` function attempts to "crack" the cipher by systematically testing all possible 10-bit keys ($2^{10} = 1024$ combinations). For each key, it encrypts the original plaintext and checks if the resulting ciphertext matches the known ciphertext.
- Key Space Exhaustion:** Due to the limited size of the S-DES key space, this brute-force loop runs quickly. The code keeps track of and displays any keys that yield matching ciphertexts, which may include the original key or other keys that coincidentally produce the same result.
- Comparison and Output:** The `main` function coordinates the overall workflow by taking user input, carrying out a standard encryption/decryption cycle, and then launching the brute-force attack to showcase the vulnerabilities that come with short key lengths.

2. Output

```

=== SIMPLIFIED DES (SDS) ===
Enter 8-bit plaintext: 11010111
Enter 10-bit key: 010000010
Invalid input. Enter exactly 10 bits (0/1): 010000010
Invalid input. Enter exactly 10 bits (0/1): 1010000010

Subkey K1: 10100100
Subkey K2: 01000011

Plaintext : 11010111
Ciphertext: 10101000
Decrypted : 11010111

Decryption successful - original plaintext recovered.

```

3. Brute-Force Output

```

Starting brute force over all 1024 possible keys...

Key 0 (binary 0000000000) -> Ciphertext: 10000100
Key 1 (binary 0000000001) -> Ciphertext: 01101101
Key 2 (binary 0000000010) -> Ciphertext: 11000110
Key 3 (binary 0000000011) -> Ciphertext: 10101111
Key 4 (binary 0000000100) -> Ciphertext: 01100001
Key 5 (binary 0000000101) -> Ciphertext: 10101101
Key 6 (binary 0000000110) -> Ciphertext: 11100011
Key 7 (binary 0000000111) -> Ciphertext: 01101111
Key 8 (binary 0000001000) -> Ciphertext: 00110100
Key 9 (binary 0000001001) -> Ciphertext: 11011001
Key 10 (binary 0000001010) -> Ciphertext: 10110110
Key 11 (binary 0000001011) -> Ciphertext: 01011011
Key 12 (binary 0000001100) -> Ciphertext: 00110101
Key 13 (binary 0000001101) -> Ciphertext: 01011001
Key 14 (binary 0000001110) -> Ciphertext: 01101111
Key 15 (binary 0000001111) -> Ciphertext: 11011011
Key 16 (binary 0000010000) -> Ciphertext: 01000001
Key 17 (binary 0000010001) -> Ciphertext: 01001000
Key 18 (binary 0000010010) -> Ciphertext: 11100011
Key 19 (binary 0000010011) -> Ciphertext: 10001010
Key 20 (binary 0000010100) -> Ciphertext: 01000100
Key 21 (binary 0000010101) -> Ciphertext: 11000101
Key 22 (binary 0000010110) -> Ciphertext: 11000110
Key 23 (binary 0000010111) -> Ciphertext: 10000111
Key 24 (binary 0000011000) -> Ciphertext: 00110101
Key 25 (binary 0000011001) -> Ciphertext: 11011100
Key 26 (binary 0000011010) -> Ciphertext: 10110111
Key 27 (binary 0000011011) -> Ciphertext: 01011110
Key 28 (binary 0000011100) -> Ciphertext: 00110100
Key 29 (binary 0000011101) -> Ciphertext: 10110101
Key 30 (binary 0000011110) -> Ciphertext: 01110110
Key 31 (binary 0000011111) -> Ciphertext: 00110111
Key 32 (binary 0000100000) -> Ciphertext: 00000100
Key 33 (binary 0000100001) -> Ciphertext: 01101101
Key 34 (binary 0000100010) -> Ciphertext: 00000110
Key 35 (binary 0000100011) -> Ciphertext: 01101111
Key 36 (binary 0000100100) -> Ciphertext: 10100001
Key 37 (binary 0000100101) -> Ciphertext: 01101101
Key 38 (binary 0000100110) -> Ciphertext: 01100011
Key 39 (binary 0000100111) -> Ciphertext: 11101111
Key 40 (binary 0000101000) -> Ciphertext: 10110100
Key 41 (binary 0000101001) -> Ciphertext: 01011001
Key 42 (binary 0000101010) -> Ciphertext: 00110110
Key 43 (binary 0000101011) -> Ciphertext: 00011011
Key 44 (binary 0000101100) -> Ciphertext: 01110101
Key 45 (binary 0000101101) -> Ciphertext: 00011001
Key 46 (binary 0000101110) -> Ciphertext: 11110111
Key 47 (binary 0000101111) -> Ciphertext: 01011011
Key 48 (binary 0000110000) -> Ciphertext: 00100001
Key 49 (binary 0000110001) -> Ciphertext: 11001000
Key 50 (binary 0000110010) -> Ciphertext: 00100011
Key 51 (binary 0000110011) -> Ciphertext: 01001010
Key 52 (binary 0000110100) -> Ciphertext: 10000100
Key 53 (binary 0000110101) -> Ciphertext: 00000101
Key 54 (binary 0000110110) -> Ciphertext: 01000110
Key 55 (binary 0000110111) -> Ciphertext: 00000111

```

```

Key 192 (binary 0011000000) -> Ciphertext: 00011000
Key 193 (binary 0011000001) -> Ciphertext: 11110101
Key 194 (binary 0011000010) -> Ciphertext: 10101000 <--- MATCH
Key 195 (binary 0011000011) -> Ciphertext: 01000001
Key 196 (binary 0011000100) -> Ciphertext: 00011000
Key 197 (binary 0011000101) -> Ciphertext: 11110100
Key 198 (binary 0011000110) -> Ciphertext: 00101000
Key 199 (binary 0011000111) -> Ciphertext: 10000100
Key 200 (binary 0011001000) -> Ciphertext: 00101010
Key 201 (binary 0011001001) -> Ciphertext: 00000011
Key 202 (binary 0011001010) -> Ciphertext: 10101000 <--- MATCH
Key 203 (binary 0011001011) -> Ciphertext: 01000001
Key 204 (binary 0011001100) -> Ciphertext: 10101010
Key 205 (binary 0011001101) -> Ciphertext: 00000110
Key 206 (binary 0011001110) -> Ciphertext: 00101000
Key 207 (binary 0011001111) -> Ciphertext: 10000100
Key 208 (binary 0011010000) -> Ciphertext: 10110101
Key 209 (binary 0011010001) -> Ciphertext: 11110100
Key 210 (binary 0011010010) -> Ciphertext: 00000101
Key 211 (binary 0011010011) -> Ciphertext: 01000100
Key 212 (binary 0011010100) -> Ciphertext: 10011000
Key 213 (binary 0011010101) -> Ciphertext: 00011001
Key 214 (binary 0011010110) -> Ciphertext: 01101000
Key 215 (binary 0011010111) -> Ciphertext: 00101001
Key 216 (binary 0011011000) -> Ciphertext: 10000111
Key 217 (binary 0011011001) -> Ciphertext: 00000110
Key 218 (binary 0011011010) -> Ciphertext: 00000101
Key 219 (binary 0011011011) -> Ciphertext: 01000100
Key 220 (binary 0011011100) -> Ciphertext: 11101010
Key 221 (binary 0011011101) -> Ciphertext: 10101011
Key 222 (binary 0011011110) -> Ciphertext: 01101000
Key 223 (binary 0011011111) -> Ciphertext: 00101001
Key 224 (binary 0011100000) -> Ciphertext: 10011000
Key 225 (binary 0011100001) -> Ciphertext: 01110101
Key 226 (binary 0011100010) -> Ciphertext: 00101000
Key 227 (binary 0011100011) -> Ciphertext: 11000001
Key 228 (binary 0011100100) -> Ciphertext: 11011000
Key 229 (binary 0011100101) -> Ciphertext: 00110100
Key 230 (binary 0011100110) -> Ciphertext: 10101000 <--- MATCH
Key 231 (binary 0011100111) -> Ciphertext: 00000100
Key 232 (binary 0011101000) -> Ciphertext: 10101010
Key 233 (binary 0011101001) -> Ciphertext: 01000011
Key 234 (binary 0011101010) -> Ciphertext: 00101000
Key 235 (binary 0011101011) -> Ciphertext: 11000001
Key 236 (binary 0011101100) -> Ciphertext: 00101010
Key 237 (binary 0011101101) -> Ciphertext: 10000110
Key 238 (binary 0011101110) -> Ciphertext: 10101000 <--- MATCH
Key 239 (binary 0011101111) -> Ciphertext: 00000100
Key 240 (binary 0011110000) -> Ciphertext: 00110101
Key 241 (binary 0011110001) -> Ciphertext: 01110100
Key 242 (binary 0011110010) -> Ciphertext: 10000101
Key 243 (binary 0011110011) -> Ciphertext: 11000100
Key 244 (binary 0011110100) -> Ciphertext: 01011000
Key 245 (binary 0011110101) -> Ciphertext: 11011001
Key 246 (binary 0011110110) -> Ciphertext: 00101000
Key 247 (binary 0011110111) -> Ciphertext: 10101001
Key 248 (binary 0011111000) -> Ciphertext: 00000111

```

```

Key 642 (binary 1010000010) -> Ciphertext: 10101000 <--- MATCH (ORIGINAL KEY)
Key 643 (binary 1010000011) -> Ciphertext: 01000101
Key 644 (binary 1010000100) -> Ciphertext: 00111100
Key 645 (binary 1010000101) -> Ciphertext: 11110000
Key 646 (binary 1010000110) -> Ciphertext: 00101000
Key 647 (binary 1010000111) -> Ciphertext: 10000000
Key 648 (binary 1010001000) -> Ciphertext: 00111010
Key 649 (binary 1010001001) -> Ciphertext: 01010011
Key 650 (binary 1010001010) -> Ciphertext: 00001010
Key 651 (binary 1010001011) -> Ciphertext: 00100111
Key 652 (binary 1010001100) -> Ciphertext: 00111010
Key 653 (binary 1010001101) -> Ciphertext: 10010110
Key 654 (binary 1010001110) -> Ciphertext: 10001010
Key 655 (binary 1010001111) -> Ciphertext: 00100110
Key 656 (binary 1010010000) -> Ciphertext: 10110001
Key 657 (binary 1010010001) -> Ciphertext: 11110000
Key 658 (binary 1010010010) -> Ciphertext: 00000001
Key 659 (binary 1010010011) -> Ciphertext: 01000000
Key 660 (binary 1010010100) -> Ciphertext: 10111100
Key 661 (binary 1010010101) -> Ciphertext: 00111101
Key 662 (binary 1010010110) -> Ciphertext: 01101000
Key 663 (binary 1010010111) -> Ciphertext: 00101001
Key 664 (binary 1010011000) -> Ciphertext: 11010111
Key 665 (binary 1010011001) -> Ciphertext: 01010110
Key 666 (binary 1010011010) -> Ciphertext: 10100111
Key 667 (binary 1010011011) -> Ciphertext: 00100110
Key 668 (binary 1010011100) -> Ciphertext: 01111010
Key 669 (binary 1010011101) -> Ciphertext: 00111011
Key 670 (binary 1010011110) -> Ciphertext: 11001010
Key 671 (binary 1010011111) -> Ciphertext: 10001011
Key 672 (binary 1010100000) -> Ciphertext: 10111100
Key 673 (binary 1010100001) -> Ciphertext: 01010101
Key 674 (binary 1010100010) -> Ciphertext: 00101000
Key 675 (binary 1010100011) -> Ciphertext: 11000101
Key 676 (binary 1010100100) -> Ciphertext: 11111100
Key 677 (binary 1010100101) -> Ciphertext: 00110000
Key 678 (binary 1010100110) -> Ciphertext: 10101000 <--- MATCH
Key 679 (binary 1010100111) -> Ciphertext: 00000000

```

```

Key 714 (binary 1011001010) -> Ciphertext: 10101000 <--- MATCH
Key 715 (binary 1011001011) -> Ciphertext: 01000101
Key 716 (binary 1011001100) -> Ciphertext: 00111100
Key 717 (binary 1011001101) -> Ciphertext: 11110000
Key 718 (binary 1011001110) -> Ciphertext: 00101000
Key 719 (binary 1011001111) -> Ciphertext: 10000000
Key 720 (binary 1011010000) -> Ciphertext: 11010111
Key 721 (binary 1011010001) -> Ciphertext: 01010110
Key 722 (binary 1011010010) -> Ciphertext: 10100111
Key 723 (binary 1011010011) -> Ciphertext: 00100110
Key 724 (binary 1011010100) -> Ciphertext: 01111010
Key 725 (binary 1011010101) -> Ciphertext: 00111011
Key 726 (binary 1011010110) -> Ciphertext: 11001010
Key 727 (binary 1011010111) -> Ciphertext: 10001011
Key 728 (binary 1011011000) -> Ciphertext: 10110001
Key 729 (binary 1011011001) -> Ciphertext: 11110000
Key 730 (binary 1011011010) -> Ciphertext: 00000001
Key 731 (binary 1011011011) -> Ciphertext: 01000000
Key 732 (binary 1011011100) -> Ciphertext: 10111100
Key 733 (binary 1011011101) -> Ciphertext: 00111101
Key 734 (binary 1011011110) -> Ciphertext: 01101000
Key 735 (binary 1011011111) -> Ciphertext: 00101001
Key 736 (binary 1011100000) -> Ciphertext: 11111010
Key 737 (binary 1011100001) -> Ciphertext: 10010011
Key 738 (binary 1011100010) -> Ciphertext: 10001010
Key 739 (binary 1011100011) -> Ciphertext: 01100111
Key 740 (binary 1011100100) -> Ciphertext: 10111010
Key 741 (binary 1011100101) -> Ciphertext: 00010110
Key 742 (binary 1011100110) -> Ciphertext: 00001010
Key 743 (binary 1011100111) -> Ciphertext: 10100110
Key 744 (binary 1011101000) -> Ciphertext: 10111100
Key 745 (binary 1011101001) -> Ciphertext: 01010101
Key 746 (binary 1011101010) -> Ciphertext: 00101000
Key 747 (binary 1011101011) -> Ciphertext: 11000101
Key 748 (binary 1011101100) -> Ciphertext: 11111100
Key 749 (binary 1011101101) -> Ciphertext: 00110000
Key 750 (binary 1011101110) -> Ciphertext: 10101000 <--- MATCH

```

```

===== BRUTE FORCE COMPLETE =====
Found 8 matching key(s):
0011000010 (decimal 194)
0011001010 (decimal 202)
0011100110 (decimal 230)
0011101110 (decimal 238)
1010000010 (decimal 642) <--- ORIGINAL KEY
1010100110 (decimal 678)
1011001010 (decimal 714)
1011101110 (decimal 750)

```

4. Observation

The code shows how Confusion and Diffusion work using S-boxes and permutations. It includes a brute-force function that connects the key space to security. With 2^{10} (or 1024) possible keys, the program can quickly test every key combination. It's important to understand that different keys can produce the same ciphertext for the same data block. This highlights the necessity of using various pairs of plaintexts and ciphertexts when trying to break the code.

5. Results

When the program runs, it processes an 8-bit message with a 10-bit key. It creates two smaller keys and produces an encrypted message. The decryption function can reverse this process to recover the original message, showing that the system works correctly. Additionally, the brute-force method can easily recover the original encryption key, demonstrating how quickly a short key can be broken through a thorough search.

6. Advantages

- **Clarity for Learning:** This code offers a clear, bit-level understanding of block cipher functionality, including key scheduling and substitution.
- **Minimal Complexity:** The lightweight design ensures rapid execution, making it ideal for classroom demonstrations.
- **Reversible Process:** The Feistel structure showcases its ability to facilitate both encryption and decryption using the same hardware or logic.
- **Verification:** The printing of subkeys allows users to manually check each step of the encryption process against theoretical calculations.

7. Disadvantages

- **Limited Key Space:** With only 1024 possible keys, the cipher offers insufficient protection against modern computing capabilities.
- **Small Block Size:** The 8-bit block size is particularly vulnerable to statistical analysis and "dictionary attacks."
- **Security Weaknesses:** This implementation primarily serves as a toy cipher; using it for real-world data might lead to serious security breaches.
- **Absence of IV or Padding:** Lack of Initialization Vectors (IVs) and padding schemes heightens the risk of pattern recognition.

8. Conclusion

The provided code effectively simulates the Data Encryption Standard (DES) architecture for educational purposes, successfully bridging cryptographic theory and practical C++ implementation. While the algorithm operates correctly, its vulnerability to brute-force attacks highlights the need for modern encryption standards like AES, which use significantly larger key sizes to ensure data security.

9. Linear & Differential Cryptanalysis on SDES

Both linear and differential cryptanalysis are efficient shortcut attacks aimed at compromising the cipher more swiftly than through brute force by exploiting its internal structure, especially the S-boxes.

Feature	Linear Cryptanalysis	Differential Cryptanalysis
Core Idea	Identifies strong linear approximations between plaintext and ciphertext bits.	Examines how specific "differences" in plaintext pairs affect the resulting ciphertext pairs.
Target Area	Primarily focuses on S-boxes to uncover linear biases in their substitutions.	Analyzes how XOR differences propagate through the Feistel rounds.
Data Requirement	Needs a large number of known plaintext-ciphertext (PT, CT) pairs.	Involves selecting specific plaintext inputs in a chosen-plaintext attack.
Objective	To formulate a linear equation $P \oplus C = K$ that holds with a probability of $\neq 0.5$.	To identify "differentials" with a high likelihood that can reveal bits of the subkey.
Relevance to S-DES	Highly effective due to the strong linear biases present in the S-boxes S_0 and S_1 .	Effective because S-DES has only 2 rounds, making differences predictable.
Complexity	Significantly narrows the search space below the 2^{10} brute-force limit.	Often identifies the key with only a handful of carefully selected plaintext pairs.
Efficiency on S-DES	Generally, requires more data but offers less control compared to differential methods.	Typically, faster than linear analysis when the attacker can manipulate the input.